### Cancer Genomics Dataset

- What features are present in the cancer genomics dataset?

The expression of different genes are the features of the cancer genomics dataset.

- What preprocessing steps did you apply (if any)?

After loading the dataset, I de-duplicated the dataset because the heart disease dataset contains multiple dupliated entries, and I might as well do the same for this dataset. However, there's no duplicated entries in this dataset.

- What initial observations did you make about the dataset (dimensionality, class balance, missingness, feature scale, etc.)?

There are lots of features in this dataset, suggesting potentially necessary feature selection / reduction.

### Model Performance (Heart Disease + Cancer Genomics)

- Which dataset was easier/harder to classify?

In my case, the cancer genomics dataset was easier to classify, as my model achieves almost perfect classification results. The same gradient boosting model achieved only 91% accuracy on the heart disease dataset after removing the duplicates from the dataset.

- Which errors were most common (especially in the cancer dataset)?

In my case, there was no error on the cancer dataset. See figure 1.

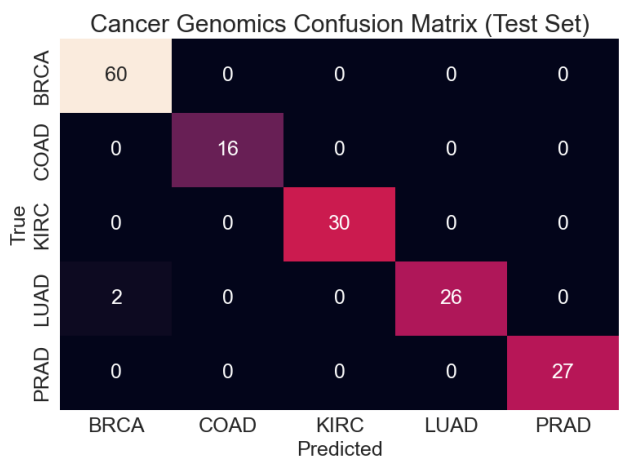


Figure 1: Cancer Confusion Matrix

### Hyperparameter Tuning (Gradient Boosting)

- Which hyperparameters did you explore (e.g., `max_depth`, `n_estimators`, `learning_rate`, etc.)?

I explored all `max_depth`, `n_estimators`, and `learning_rate`. On top of these three, I also explored `subsample`, `min_sample_leaf`, and `max_features` by the recommendation of generative AI.

- What range of values did you test?

See table 1.

- Which configuration performed best on each dataset, and why do you think it worked well?

| **Hyperparameter** | **Values Tested** |
|---|---|
| Maximum tree depth (`max_depth`) | {1, 2, 3} |
| Number of estimators (`n_estimators`) | {50, 100, 200, 400} |
| Learning rate (`learning_rate`) | {0.01, 0.05, 0.1, 0.2} |
| Subsample ratio (`subsample`) | {0.6, 0.8, 1.0} |
| Minimum samples per leaf (`min_samples_leaf`) | {1, 2, 5} |
| Maximum features (`max_features`) | {`None`, `sqrt`, `log2`} |

Table 1: Hyperparameter search space used for gradient boosting model selection.

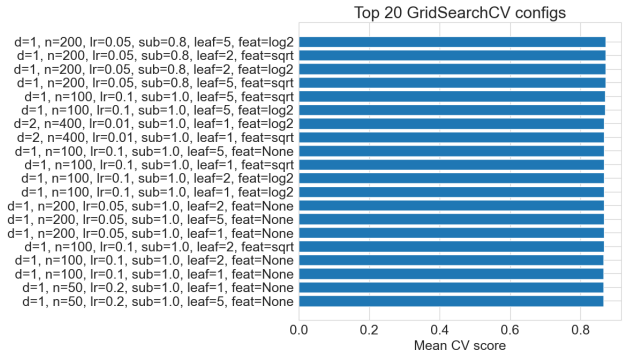


Figure 2: Grid Search Plot

The configuration that worked the best on the heart dataset was `'learning_rate': 0.01, 'max_depth': 3, 'max_features': 'sqrt', 'min_samples_leaf': 5, 'n_estimators': 200, 'subsample': 0.6`. I think this works very well because it is a sweet spot in the bias-variance tradeoff (especially the number of estimators, given that more or less estimators would not improve the performance).

By the completion of this writeup, I still have not got the tuning results from the cancer genomics dataset. Run my notebook if this is necessarily needed for grading.

- Provide grid search table or plot.

See figure 2. They don't look much different to me.

### Feature Importance and Interpretation

- For gradient boosting: report the top features by importance (and include a figures/table/plot).

For the heart disease dataset, the most important feature is `cp`, which is the type of chest pain. For obvious reasons this feature is very important, as I already discussed in the writeup of the previous assignment. This is followed by `ca` and `thal`. See table 2.

For the cancer genomics dataset, the most important features are a bunch of genes, which I don't think are very informative to be displayed here but see table 3 for details:

- Compare feature importance to HW1 feature selection (for example: using Lasso to identify important features). Do the two methods agree on what features matter most? Why or why not?

The feature importance for gradient boosting agrees with the feature selection in HW1 for the most part. I believe the similarity is most definitely due to the dataset's nature: any model performing well on the dataset should value the most important features. For the differences, I would argue that they are due to the model nature: some features are better resolved with gradient boosting, whereas logistic regression is a linear model with only one shot.

| Feature Name | Importance |
|---|---|
| cp | 0.327901 |
| ca | 0.126882 |
| thal | 0.111340 |
| thalach | 0.082485 |
| oldpeak | 0.074100 |
| age | 0.068439 |
| chol | 0.049616 |
| exang | 0.041964 |
| sex | 0.040327 |
| trestbps | 0.036792 |
| restecg | 0.020693 |
| slope | 0.016694 |
| fbs | 0.002767 |

Table 2: Feature importances from the trained gradient boosting model on the heart disease dataset.

| Gene | Importance |
|---|---|
| gene_18746 | 0.144764 |
| gene_15898 | 0.095321 |
| gene_15895 | 0.093105 |
| gene_12983 | 0.089372 |
| gene_3523 | 0.078836 |
| gene_203 | 0.076656 |
| gene_17801 | 0.072281 |
| gene_9176 | 0.067083 |
| gene_7964 | 0.066929 |
| gene_6733 | 0.053245 |

Table 3: Top 10 most important gene features identified by the gradient boosting model.

**Comparison to HW1** In my case, the Logistic Regression model trained with top-100 features also achieved 100% accuracy and 1.0 F1 score. I highly suspect that the dataset is broken, as supported by EdStem post #37. Considering that multiple students have reported this issue with the cancer genomics dataset, I would keep my results reported as-is.

**Discussions** It is reasonable to infer that the cancer genomics classification task is genuinely easy. The gradient boosting model performs very well on both tasks with proper tuning, suggesting that the model is very capable of doing both tasks.

**Collaboration** This project is done independently with the help of generative AI (ChatGPT 5.2, specifically).